Egyenesvonalú (lineáris) adatszerkezetek

Tömb: azonos típusú elemekből álló, sorosan tárolt adatcsoport. Amennyiben hossza előre ismert, statikus tömbnek nevezik, ha hossza a program futása során (runtime) változik, dinamikus tömbnek, vagy listának nevezzük.

Többdimenziós tömbök: akár n-dimenzióig bővíthetőek, minden dimenzióhoz tartozik egy index, egy elemre az egyes dimenziók indexeiből adódó pozíció meghatározásával lehet.

Rekordszerkezet

- Több, esetleg különböző részekből összetett adattípus
- ~Osztály, függvények nélkül, csak látható tagokkal
- Akkor véges, ha az összetevők végesek, és ha nem rekurzív

Láncolt lista: az egyes elemek külön–külön tartalmaznak pointer(eke)t, mellyel megvalósítható az elemek lineáris rendezettsége (típusai: egyszeresen/kétszeresen láncolt/ciklikus)

Vermek(FIFO és LIFO)

LIFO (Last In, First Out): Amit legelsőnek raktunk bele azt utoljára tudjuk csak kivenni. (stack)

FIFO (First In, First Out): Amit elsőnek rakunk be, azt elsőként is tudjuk kivenni. (queue)

Vermek megvalósítása:

```
public class verem {
int max = 100;
int tomb[] = new int[max];
int teteje=0;
public boolean ures() {
 return (teteje == 0);}
public void beszur(int elem) {
 tomb[teteje++]=elem;}
public int kivesz() {
 return (tomb[--teteje]);}
public boolean tele() {
 return (teteje==max);
}
```

```
public class sor {
    int max = 100;
    int tomb[] = new int[max];
    int feje=0;
    int farka=0;
    boolean ures=true;
    boolean tele=false;
     public boolean ures() {return (ures);}
     public boolean tele() {return (tele);}

    public void beszur(int elem) {
     tomb[farka]=elem;
     ures = false;
     if (farka<max-1) farka++;
      else farka=0;
     if (farka == feje) tele = true;}
    public int kivesz() {
     int elem;
     elem = tomb[feje];
     tele = false;
     if (feje<max-1) feje++;
      else feje = 0;
     if (feje == farka)
      ures = true;
      return (elem);
 }
```

Bináris kupac (heap) adattípus

- Bináris fa, amelyben minden csomópont értéke nagyobb mindegyik leszármazottjáénál (=*kupactulajdonság*)
- Magasság: a csúcsból a levélig vezető leghosszabb út
- Kiegyenlített: csak az utolsó réteg lehet nem teljes
- Ábrázolás: vektorban, rétegenként: Minden réteg mérete=korábbi rétegek+1. Minden réteg első elemindexe=2^h

Kupacrendezés algoritmusa (műveletigénye *O(n log n)*)

(Kupacolás: a rossz helyen levő elemeket lefelé görgeti, mindig a nagyobb gyerek felé.)

Működése során felépíti a kupacot, majd egyesével kiemeli a gyökérelemet, ami a kupac definíciója miatt a legnagyobb/legkisebb elem lesz. A kupac legalsó szintjének jobb szélső elemével cserélünk, és az új jobb szélső elemet nem tekintjük a kupac részének. Helyreállítás: az eredeti jobb szélső elemet a gyökérbe helyezzük, majd süllyesztjük. A süllyesztés a szülő elemet mindig a nagyobbik, illetve a jobb oldali gyerekkel cseréli, ha van ilyen, és nagyobb a szülőnél. Az algoritmus ezután mindig eltávolítja a következő legnagyobb, azaz a gyökérben található elemet, és helyreállítja a kupactulajdonságot.

Szoftverfelépítmény: egy adott rendszer struktúrája, az egyes komponensek, azok látható tulajdonságai/kapcsolatai.
Szoftverfelépítmény minta: „ötlet, ami hasznosnak bizonyult egy gyakorlati helyzet megoldásában, és hasznos lehet mások számára is", útmutató egy rendszer felépítésére, leírja egy szoftverrendszer legfontosabb strukturális felépítését, definiálja a részrendszereket és funkcióikat, különböző szabályokat és ajánlásokat tartalmaz. Jellemzően tartalmazza: nevét, kapcsolódó problémaleírást, használat előfeltételeit, erősségeit (pl. érthetőség, újrafelhasználhatóság), vele elérhető célt, példákat, hasonló mintákat, ismert használati helyeit.
Szoftverfelépítmény stílus: hasonló szerkezetű rendszerek egy családjának leírása, leírja az összetevőket és a hozzátartozó kapcsolóelemeket (eljáráshívások, adatbázis-protokollok, csövek).

Elemzési minták: egyfajta koncepcionális minták, jellemzően egy valós helyzet megoldásának valamilyen leírása. Összetartozó, általános objektumok egy csoportja, amit adatdefiníciókat, eljárásokat és a közöttük elvárt együttműködéseket tartalmaznak.
Tervezési minták: a gyakran előforduló programozási feladatokra adható általános, újrafelhasználható megoldások. Előnyük, hogy a gyakori problémákra adott válaszok meghatározásával lerövidítik a tapasztalatszerzési és tervezési időt.
Négyek bandája: egy könyv (Programtervezési minták) négy szerzője, akik ebben írták le a szoftvertervezés jellemző problémáit. Az általuk meghatározott tervezési mintákat három nagy csoportba sorolták:
- Létrehozói minták: objektumok létrehozásnak módjával foglalkoznak, hogy milyen módon hozzuk létre az objektumokat a helyzetnek megfelelő módon.

- o Singleton: olyan objektum, amiből garantáltan egyetlen példány létezhet csak egy rendszerben. Rendszerint tartalmazza saját egyetlen példányát.
- o Abstract factory: egy feladathoz szükséges objektumot nem a saját konstruktorával hozzuk létre, hanem a factory egy függvényhívásával. Egy felületet ad akár összetartozó, akár független objektumok létrehozásához, anélkül, hogy megadnánk a konkrét osztálytípust.
- o Builder: elválasztja egy komplex objektum létrehozását a reprezentációjától, így ugyanaz a létrehozó eljárás képes különböző reprezentációkat is létrehozni.
- o Prototype: megad egy objektum prototípust, amiből aztán másolással lehet létrehozni újabb példányokat.

- Szerkezeti minták: objektum összetételekre épülnek, elősegítik a tervezést azzal, hogy egy egyszerű utat nyitnak az egyes példányok közti kapcsolatok felderítéséhez.

- o Adapter: burkoló osztály, egy osztály felületét átalakítja olyan módon, ahogy azt egy másik objektum elvárja.
- o Bridge: elválasztja a definíciót a megvalósítástól, így ezek külön tudnak változni egymástól.
- o Composite: több független objektumból képez egy fa-struktúrájú összetett objektumot, amit aztán, mint egyetlen objektumot lehet tovább kezelni.
- o Decorator: egy osztályhoz dinamikusan ad hozzá új műveleteket, kibővítve a funkcionalitást.
- o Facade: több interface számára biztosít egy egységes, egyszerűsített felületet.
- o Proxy: egy felület egy másik osztály felé, amin keresztül lehet kezelni az adott osztályt. Jellemzően egy osztályhoz több proxy tartozhat.

- Viselkedési minták: az objektumok közötti kommunikációt leíró minták. Néhány példa:

- o Command: objektumba „csomagol" egy kérést a paramétereivel együtt.
- o Iterator: lehetőséget ad egy összetett objektum egyes elemeinek szekvenciális elérésére anélkül, hogy a belső struktúrával foglalkozna.
- o Mediator: egy objektumot definiál, amibe „becsomagolja", hogy más objektumok milyen módon működnek együtt. Egy felületet biztosít más felületeknek egy alrendszerben.
- o State: egy módja annak, ahogy egy objektum futásidőben képes legyen részben megváltoztatni a saját típusát.